

Chapter 37:

好的，我们来详细分析一下在 UE 中应用 GameplayEffect (GE) 的规范编码流程，并深入解析 AuraEffectActor.cpp 中 ApplyEffectToTarget 函数的每一个步骤。

应用 GameplayEffect 的规范编码流程

在 Unreal Engine 的 Gameplay Ability System (GAS) 插件中，应用一个 GameplayEffect (游戏效果) 是一个核心操作。一个规范、标准的流程不仅能保证功能正确实现，还能确保代码的健壮性、可扩展性和网络同步的正确性。

核心流程如下：

1. 获取目标 AbilitySystemComponent (ASC):

- 任何想要接收 GameplayEffect 的 Actor 都必须拥有一个 UAbilitySystemComponent 组件。
- 因此，第一步总是要从目标 Actor 身上获取这个组件。如果获取失败，后续所有操作都无法进行。

2. 创建 GameplayEffectContext (效果情景):

- FGameplayEffectContext 是一个结构体，它包含了关于 GE 的“元数据”，即“谁、什么、在哪里、为什么”等信息。
- 例如，是谁施加了这个效果 (Instigator)、效果来源于哪个 Actor (EffectCauser)、技能或物体的源对象 (SourceObject) 等。
- 正确填充 Context 对于后续的计算 (如伤害计算中的暴击、闪避判断)、效果追溯和调试至关重要。

3. 创建 GameplayEffectSpec (效果规格):

- FGameplayEffectSpec 是 GameplayEffect 的“实例化对象”。直接应用 UGameplayEffect 类是无效的，必须先创建一个 Effect Spec。
- Effect Spec 包含了从 UGameplayEffect 类模板中创建的所有信息，并可以动态修改，例如：
 - 设置效果的等级 (Level)。
 - 动态分配 SetByCaller 类型的 Magnitude (效果数值)。
 - 添加或修改 Tag。
- 这个步骤将一个通用的效果“蓝图” (UGameplayEffect) 变成了一个具体

的、准备施加到目标身上的“实例” (FGameplayEffectSpec)。

4. 应用 GameplayEffectSpec 到目标 ASC:

- 最后一步，调用目标 ASC 的 ApplyGameplayEffectSpecToSelf 或 ApplyGameplayEffectSpecToTarget 函数，将创建好的 Effect Spec 应用到组件上。
- ApplyGameplayEffectSpecToSelf 通常在 ASC 内部（例如在一个 GameplayAbility 里）调用，目标就是自身。
- ApplyGameplayEffectSpecToTarget 则用于将效果应用到另一个 ASC 上。
- 一旦应用，GAS 系统会自动处理效果的持续时间、周期性触发 (Period)、属性修改 (Modifiers)、Tag 增删等所有逻辑，并处理好服务器与客户端之间的同步。

ApplyEffectToTarget 函数详解

现在，我们来逐行分析 AuraEffectActor.cpp 中的 ApplyEffectToTarget 函数，看看它是如何遵循上述规范流程的。

C++

```
// AuraEffectActor.cpp
```

```
void AAuraEffectActor::ApplyEffectToTarget(AActor* Target,
TSubclassOf<UGameplayEffect> GameplayEffectClass)
{
    // 步骤 1: 获取目标的 AbilitySystemComponent

    UAbilitySystemComponent* TargetASC =
    UAbilitySystemBlueprintLibrary::GetAbilitySystemComponent(Target);

    if(TargetASC == nullptr) return; // 防御性编程，如果目标没有 ASC，则直接返回

    // 步骤 2: 检查传入的 GameplayEffect 类是否有效

    check(GameplayEffectClass); // 确保传入的效果蓝图是有效的，否则在开发阶段就
    报错
```

```

// 步骤 3: 创建并填充 GameplayEffectContext

FGameplayEffectContextHandle EffectContextHandle =
TargetASC->MakeEffectContext();

EffectContextHandle.AddSourceObject(this); // 将当前 Actor (AuraEffectActor) 设置
为源对象


// 步骤 4: 创建 GameplayEffectSpec

const FGameplayEffectSpecHandle EffectSpecHandle =
TargetASC->MakeOutgoingSpec(GameplayEffectClass, 1.0f, EffectContextHandle);


// 步骤 5: 将 Effect Spec 应用到目标 ASC

TargetASC->ApplyGameplayEffectSpecToSelf(*EffectSpecHandle.Data.Get());

}

```

步骤 1 & 2: 获取组件与检查有效性

- **UAbilitySystemComponent* TargetASC =**
UAbilitySystemBlueprintLibrary::GetAbilitySystemComponent(Target);
 - **功能:** 这是应用效果的第一步也是最关键的一步。它通过一个静态库函数 `GetAbilitySystemComponent` 尝试从传入的 `Target Actor` 上获取其 `UAbilitySystemComponent`。这个函数会自动检查 `Actor` 本身以及其组件，看是否有 `ASC`。
 - **必要性:** `GAS` 的所有核心功能，包括属性 (`Attributes`)、能力 (`Abilities`) 和效果 (`Effects`)，都由 `ASC` 管理。没有 `ASC`，就无法应用 `GE`。代码中的 `if(TargetASC == nullptr) return;` 是一种健壮性检查，防止在无效目标上执行后续操作而导致程序崩溃。
- **check(GameplayEffectClass);**
 - **功能:** `check` 是一个断言宏。它会检查括号内的表达式是否为真。如果为假（在这里是 `GameplayEffectClass` 为空指针），程序会立即中断并报错。这通常用于在开发阶段捕捉那些本不应该发生的逻辑错误。
 - **必要性:** 如果不检查，而在一个空的 `GameplayEffectClass` 上创建 `Effect Spec`，程序同样会在后续步骤中崩溃。使用 `check` 可以在问题发生的第

一时间就定位到错误源头，即调用 `ApplyEffectToTarget` 的地方没有正确传入一个有效的 GE 类。

步骤 3: 创建和配置 Effect Context

- **FGameplayEffectContextHandle EffectContextHandle = TargetASC->MakeEffectContext();**
 - **功能:** 从目标的 ASC 创建一个 FGameplayEffectContextHandle。Handle（句柄）是一种智能指针，可以方便地管理 Context 对象的生命周期。MakeEffectContext 会预先填充一些基础信息，比如 Instigator（发起者）等。
 - **必要性:** EffectContext 是效果的“身份证”，记录了效果的来源和背景信息。即使你不需要自定义额外信息，创建一个默认的 Context 也是规范流程的一部分。后续的很多高级功能，比如在伤害计算公式（Calculation Class）中获取攻击者的数据，都依赖于 Context 中存储的信息。
- **EffectContextHandle.AddSourceObject(this);**
 - **功能:** 在刚刚创建的 Context 中添加一个“源对象”（Source Object）。这里的 this 指的是 AAuraEffectActor 这个 Actor 实例本身。
 - **必要性:** 这一步非常重要，它明确了“是什么东西产生了此效果”。例如，一个火球击中目标，火球 Actor 就是 SourceObject；一个光环持续对周围敌人造成伤害，光环 Actor 就是 SourceObject。它将效果与游戏世界中的一个具体对象关联起来，对于调试、日志记录以及实现特定游戏逻辑（例如“只有来自‘神圣祭坛’的治疗效果才能驱散此诅咒”）至关重要。

步骤 4: 创建 Effect Spec

- **const FGameplayEffectSpecHandle EffectSpecHandle = TargetASC->MakeOutgoingSpec(GameplayEffectClass, 1.0f, EffectContextHandle);**
 - **功能:** 这是将 UGameplayEffect 类（模板）转化为 FGameplayEffectSpec（实例）的核心步骤。
 - `TargetASC->MakeOutgoingSpec(...)`: 表示由 TargetASC 的拥有者（或其代理）创建一个“即将施放出去”的效果规格。
 - `GameplayEffectClass`: 指定了要使用哪个 GE 蓝图作为模板。

- 1.0f: 这是效果的等级 (Level)。很多 GE 的数值会随着等级变化，这里硬编码为 1 级。
- EffectContextHandle: 将前面创建的 Context 与这个 Spec 关联起来。
- **必要性:** 你不能直接应用一个 GE 类，必须通过 Effect Spec。这个 Spec 是一个包含了所有最终计算数值、标签、持续时间等信息的完整数据包，是真正被应用到目标身上的东西。这一步允许你在应用前最后一次动态修改效果（例如，根据玩家的某个属性来决定 Spec 的等级），提供了极大的灵活性。

步骤 5: 应用 Effect Spec

- **TargetASC->ApplyGameplayEffectSpecToSelf(*EffectSpecHandle.Data.Get());**
 - **功能:** 调用目标 ASC 的函数，将创建好的 Effect Spec 应用到它自己身上。
 - EffectSpecHandle.Data.Get(): 从句柄中获取底层的 FGameplayEffectSpec 对象的指针。
ApplyGameplayEffectSpecToSelf 需要的是一个原始的 Spec 引用，而不是句柄。
 - **必要性:** 这是整个流程的最后一步，是真正触发效果的调用。一旦执行，GAS 系统就会接管一切：
 - 如果 GE 是即时的 (Instant)，它会立即修改目标的属性。
 - 如果 GE 是持续的 (Duration) 或无限的 (Infinite)，它会被添加到 ASC 的激活效果列表中，并在其生命周期内持续影响目标。
 - 服务器会自动将此效果同步给所有相关的客户端，保证状态一致。

总结

AuraEffectActor 中的 ApplyEffectToTarget 函数是一个非常经典和规范的 GAS 效果应用示例。它完美地遵循了“**获取 ASC -> 创建 Context -> 创建 Spec -> 应用 Spec**”这一黄金流程。每一步都有其明确的功能和存在的必要性，共同构成了一个健壮、可扩展且网络友好的效果施加系统。理解这个流程是掌握和使用 GAS 的关键。

GE:

总览

GameplayEffect (GE) 是一个数据资产，它的核心作用是定义一个**“游戏效果”**。这个效果可以是对角色属性的修改、状态的改变、甚至是视觉特效的触发。下面我们来逐一解析界面中的每个卷展栏。

1. 状态 (Status)

这个分类下的功能与游戏的实际运行逻辑无关，纯粹是为了方便开发者在编辑器中进行管理和辨识。

- **编辑器状态文本 (Editor Status Text)**
 - **详细作用:** 这是一个备注字段。您可以在这里输入任何描述性文本，它会显示在编辑器的一些列表视图中。这不会以任何方式影响游戏。
 - **使用情形:** 当您的项目中有成百上千个 GE 时，一个好的命名和备注习惯至关重要。例如，您可以将一个造成火焰伤害的 GE 备注为“DOT-火焰伤害-每秒 5 点-持续 10 秒”，这样您无需点开 GE 的每一个细节，就能快速了解它的作用。
-

2. 持续时间 (Duration)

这个分类定义了 GE 的生命周期，即它将存在多久。

- **持续时间策略 (Duration Policy)**
 - **详细作用:** 这是 GE 最基本的行为模式，决定了效果的类型。
 - **即时 (Instant):** 效果在应用的那一帧立即执行其“Gameplay 效果”卷展栏中的所有逻辑（例如，修改属性），然后立刻被销毁。
 - **拥有持续时间 (Duration):** 效果在应用后会持续一段明确的时间（由下面的“持续时间幅度”定义），到期后自动移除。
 - **无限 (Infinite):** 效果会一直存在，直到被另一个 GE 或 C++ 代码逻辑手动移除。
 - **使用情形:**

- 即时: 适用于所有一次性事件, 如火球术的直接伤害、喝下小红瓶的瞬间回血、法力药水的即时法力恢复。
 - 拥有持续时间: 适用于绝大多数的 Buff (增益) 和 Debuff (减益)。例如一个持续 30 秒的“英勇祝福”Buff, 或一个持续 8 秒的“中毒”Debuff。
 - 无限: 适用于角色的光环技能 (如“惩戒光环”, 只要技能开启就一直存在)、永久性的剧情诅咒 (需要特殊条件解除)、或者角色的战斗姿态 (如“防御姿态”)。
- **持续时间幅度 (Duration Magnitude)**
 - **详细作用:** 当“持续时间策略”为拥有持续时间时, 此参数才有效。它定义了效果具体的持续秒数。通常可以使用“可缩放浮点数(Scalable Float)”来让持续时间根据 GE 的等级 (Level) 变化。
 - **使用情形:** 一个“减速”技能, 1 级时可能持续 3 秒, 升到 2 级后变为持续 4.5 秒。这个随等级变化的数值就在这里配置。
 - **Period (周期)**
 - **详细作用:** 当“持续时间策略”为拥有持续时间或无限时, 此参数才有效。它让 GE 可以周期性地、重复地触发效果。每隔 Period 设定的秒数, GE 的执行(Executions)部分就会运行一次。
 - **使用情形:** 这是实现“持续性伤害(DOT)”和“持续性治疗(HOT)”的核心。例如:
 - **中毒效果:** 设置持续时间为 8 秒, Period 为 1 秒。这样, 在 8 秒内, 目标会每秒都受到一次中毒伤害。
 - **再生 Buff:** 设置持续时间为 20 秒, Period 为 2 秒。这样, 在 20 秒内, 角色会每 2 秒恢复一次生命值。
-

3. Gameplay 效果 (Gameplay Effect)

这个分类定义了 GE 的**核心功能**, 即“这个效果到底做了什么”。如我们之前所确认, 它采用**组件化**的设计, 您需要通过点击组件(Components)旁的“+”号来添加具体的效果。

- **组件 (Components)**
 - **详细作用:** 这是一个效果容器。GE 的所有实际作用, 无论是修改属性、授予标签还是执行复杂计算, 都作为一种“组件”被添加到这里。

- **使用情形:** 一个复杂的“狂暴”Buff, 可能需要添加多个组件来实现:
 1. 添加一个修饰符组件, 用于增加攻击力。
 2. 添加第二个修饰符组件, 用于降低防御力。
 3. 添加一个授予标签组件, 赋予角色 State.Rage 标签, 使其免疫恐惧效果。
- **可添加的组件类型 (展开“组件”后):**
 - **修饰符 (Modifiers)**
 - **详细作用:** 最常用的组件。用于直接修改角色的属性 (Attribute)。每个修饰符都由三部分构成: 要修改的**属性** (如生命值)、**操作类型** (加、乘、除、覆盖) 和**量值** (修改多少)。
 - **使用情形:**
 - **加法:** 喝药水, 生命值 增加 100。
 - **乘法:** 获得“急速”Buff, 攻击速度 乘以 1.2 (即提升 20%)。
 - **覆盖:** 变身成巨人, 模型缩放比例 覆盖为 2.0。
 - **执行 (Executions)**
 - **详细作用:** 用于处理无法用简单数学公式描述的复杂逻辑。它指向一个自定义的 C++类, 在这个类里, 你可以访问攻击方和防御方的所有数据 (属性、标签等) 来计算最终结果。
 - **使用情形:** 所有复杂的伤害计算都应该用它。例如, 一个“背刺”技能的伤害, 需要根据“攻击者的敏捷”、“目标的护甲”、“是否暴击”、“目标是否处于未察觉状态”等多个因素来计算, 这些逻辑就写在执行的 C++类里。
 - **标签相关组件 (Tag-related Components)**
 - **详细作用:** 用于管理 GameplayTag, 是实现角色状态变化的核心。
 - **授予标签 (Granted Tags):** 当 GE 生效时, 赋予目标一个或多个标签。GE 移除时, 标签也随之移除。
 - **移除带有标签的 GE:** 当此 GE 应用成功后, 会立即移除目标身上所有带有指定资产标签(Asset Tag)的其他 GE。

- **使用情形:**
 - **授予标签:** 一个“冰冻”法术的 GE，会授予目标 State.Debuff.Frozen 标签。角色的移动和施法系统会检查这个标签，如果存在，则禁止相应操作，从而实现“冰冻”状态。
 - **移除 GE:** 一个“净化术”的 GE，其核心作用就是添加一个“移除带有 Debuff.Poison 标签的 GE”组件，从而实现解毒效果。
-

4. Gameplay Cues

这个分类是 GE 的**表现层**，负责将逻辑效果与视觉、听觉（VFX/SFX）表现关联起来。

- **详细作用:** GE 本身不负责播放特效或声音。它只在特定时机（如被应用时、持续期间、被移除时、每次执行时）广播一个 GameplayCue 标签（例如 GameplayCue.Damage.Fire）。然后，一个专门的蓝图或 C++ 类 GameplayCueNotify 会监听这些标签，当监听到对应的标签时，它才负责去播放粒子特效、播放声音、产生镜头震动等。这实现了逻辑与表现的解耦。
 - **使用情形:**
 - 一个火焰 Buff 的 GE，在 Gameplay Cues 里设置 WhileActive（持续期间）时广播 GameplayCue.Buff.OnFire。对应的 GameplayCueNotify 蓝图则会在角色身上持续播放火焰特效。
 - 一个挥砍技能的伤害 GE，在 OnExecute（执行时）广播 GameplayCue.Damage.Slash。对应的 GameplayCueNotify 就会在受击者身上播放刀痕特效和“噗嗤”的音效。
-

5. 堆叠 (Stacking)

这个分类定义了当**同一个 GE** 被重复应用到同一个目标上时的行为。

- **详细作用:** 控制效果的层数、持续时间刷新规则等。
 - 堆叠样式 (Stacking Type): 不堆叠(None)或按目标聚合(Aggregate by Target)。
 - 堆叠限制计数 (Stack Limit Count): 效果最多能叠多少层。
 - 堆叠持续时间刷新策略: 叠加上新的一层时，是刷新整个效果的持续时

间，还是不影响。

- 堆叠过期策略: 当持续时间到了，是清除所有层数，还是只移除一层。

- 使用情形:

- **流血 Debuff**: 堆叠限制: 5 层, 刷新策略: 应用成功时刷新。这样敌人每被击中一次, 流血效果就叠加一层 (伤害更高), 并且整个流血的持续时间会被重置, 让敌人一直流血。
- **增益药水**: 堆叠样式: 不堆叠。确保玩家不能通过连喝 5 瓶力量药水获得 5 倍的力量加成。
- **杀意 Buff**: 堆叠限制: 10 层, 过期策略: 移除单个堆栈。角色每次击杀敌人获得一层杀意, 提升攻击力。如果在一定时间内没有再次击杀, Buff 不会完全消失, 而是掉落一层, 给玩家保持高层数的压力。